

Configurable Static Type Checking for Forth

Jaanus Pöial
Tallinn University of Technology

Abstract

Forth stack comments document the cells consumed and produced by a word. Ordinary comments are not checked. They also do not record how stack words move or copy individual cells. This paper presents a configurable source-level checker. It represents each phrase by a typed input/output stack contract, called a stack effect. A profile supplies the type hierarchy and primitive word effects. Symbolic indices record the flow introduced by words such as `DUP`, `SWAP`, and `OVER`.

The analysis uses three partial operations. Composition handles word sequences. Branches keep only facts that hold on both paths. A finite one-pass-versus-two-pass comparison handles repetition. Nine small Forth definitions illustrate these operations. They cover type refinement, cell identity, hidden stack prefixes, branch merging, and local loop stability. The checker reproduces all nine reported effects without executing concrete values. Its conclusions depend on the supplied primitive specifications. The branch summary also forgets path-specific facts. The loop summary does not cover every possible iteration count.

Keywords: Forth; static type checking; stack effects; symbolic execution; subtyping; concatenative languages.

1 Introduction

A Forth word is a named operation. Here, a cell is one single-cell stack item. The data stack is the ordered sequence of those items. A word normally receives operands from that stack and returns its results to the same stack. The conventional notation

```
+   ( n n -- n )
```

places the top of stack on the right and says that `+` consumes two numbers and produces one. Such comments are valuable interfaces, but a Forth system usually consumes them as source text rather than treating them as proof obligations. A comment can therefore be missing, stale, or semantically stronger than the implementation.

Consider the familiar definition

```
: SQUARED  ( n -- n^2 )  DUP * ;
```

The checker can derive `( n - n )` from the declarations of `DUP` and `*`. It can verify the number and identity of the cells used by the phrase. It does not prove the mathematical assertion that the result is the square of the input. This distinction—typed stack consistency rather than arbitrary functional correctness—sets the scope of the method.

The checker reads three text inputs:

1. a file that defines types, aliases, and ordering relations;
2. a file that defines word effects and source-reading rules;
3. a Forth source file whose colon definitions are to be inferred.

A *primitive word* is a word whose effect comes directly from the second file. A *colon definition* begins with `:` and ends with `;`. The word that it defines receives an inferred effect from the source between those delimiters. The checker has Java and Gforth implementations. Both consume the same three files. An analysis dictionary maps each known word to one stack effect. At `;`, the inferred body effect is added to this dictionary. The checker does not execute the Forth definition.

The design builds on earlier work on stack effects and symbolic type variables [4, 5, 6, 7, 8]. This paper gives a self-contained account of the operations used by the nine examples. It explains four points:

- how a later word can constrain an earlier input;
- how two branch effects are combined;
- why an effect displays only the top cells that it touches;
- why the loop check compares one pass with two passes.

Sections 2–4 define the complete analysis used here. Sections 5–7 derive all nine results. Appendices A–C reproduce the source and every profile entry needed for those derivations. No other project document is required.

2 Typed Symbolic Stack Effects

2.1 Types and compatibility

Let T be a finite set of abstract cell types with a precision order $\preceq$. The relation $t \preceq u$ means that t is more specific than u . A value of type t may therefore be used where u is required. This order is the profile’s subtype relation. The Forth-2012-like profile used for the examples contains, among other edges, the transitive chains

$$\text{a-addr} \preceq \text{c-addr} \preceq \text{addr} \preceq \text{u} \preceq \text{n} \preceq \text{x},$$

$$\text{char} \preceq \text{+n} \preceq \text{u} \preceq \text{n} \preceq \text{x}, \quad \text{flag} \preceq \text{x}.$$

The flag type is deliberately separate from the numeric chain in this profile. Another profile may classify flags as numbers. The hierarchy is therefore a policy for one target vocabulary. It is not a universal claim about Forth cells. Names such as `x`, `X`, and `cell` may be aliases for one type.

Two types are *comparable* when one is below the other in $\preceq$. Comparable types are compatible. Matching them retains the more specific type:

$$\text{match}(t, u) = \begin{cases} t, & t \preceq u, \\ u, & u \preceq t, \\ \text{undefined}, & \text{otherwise.} \end{cases}$$

The implementation does not search for a third common subtype when t and u are incomparable. An undefined match is a type clash.

2.2 Symbols, effects, and the implicit frame

A symbolic cell is a type with an optional positive identity index, written $t[i]$. Equal indices denote the same abstract origin. Different indices do not claim that the concrete values differ. Every unindexed occurrence starts as an independent symbol. An unindexed output is therefore a fresh abstract result. Indices exist only in the checker. They are not addresses or runtime tags.

The declarations of three stack words illustrate the notation:

```
DUP   ( x[1] -- x[1] x[1] )
SWAP  ( x[2] x[1] -- x[1] x[2] )
OVER  ( x[2] x[1] -- x[2] x[1] x[2] )
```

Before each word occurrence is analyzed, its local indices receive fresh numbers. Two primitive calls therefore cannot share an index by accident. After analysis, *normalization* rennumbers the surviving indices from one upward. Renumbering changes only their printed names, not their relationships.

An effect $e = (L - -R)$ contains two finite symbolic stacks, with their tops on the right. A stack *suffix* is the group of top cells displayed by the effect. Deeper cells form an implicit *frame*. For every frame F , the contract acts as

$$F L \mapsto F R.$$

For example, $(\text{a-addr} - x)$ for $\text{\textcircled{a}}$ does not require the entire data stack to contain one cell. It requires only that the top cell be an aligned address; any deeper F passes through unchanged. The identity effect is $(-)$. Write ϵ for the empty stack. A clash is a separate failure result and must not be confused with the identity effect. The set of symbolic effects is the checker's *abstract domain*. An effect operation is *partial* when some arguments produce no effect. In those cases, this checker reports a clash.

3 The Three Effect Operations

3.1 Sequential composition

Let $p = (L_p - -R_p)$ and $q = (L_q - -R_q)$. The expression $p \cdot q$ means: apply p , then apply q . Composition compares the top output of p with the top input of q . It then works inward. For every compatible pair, the checker:

1. selects the more specific boundary type;
2. creates a fresh shared symbol of that type;
3. substitutes it for both old symbols throughout the working effects;
4. removes the matched output/input pair and continues.

If p runs out of outputs first, q still needs cells from the implicit frame. Those cells become inputs of the composite. If q runs out of inputs first, unused outputs of p remain below the outputs of q . This is exactly the effect of applying the two suffix transformations in order.

Substitution through the whole effect is essential. A boundary constraint must reach every copy with the same index. A later $\text{\textcircled{a}}$ can therefore refine a cell that passed through **SWAP** and **DUP**. Incompatible types make composition undefined. A word sequence is reduced by repeated composition from left to right. The empty sequence has the identity effect. This operation extends earlier untyped stack-effect composition [4].

3.2 Common branch contracts

Alternative paths use a second partial operation, written $p \sqcap q$. Call it the *common contract*. It contains only facts justified for both paths. It does not store a separate effect for each path. Positions are first aligned from the top of each stack.

At aligned inputs, the operation keeps the more specific comparable type. The input to the complete conditional must satisfy both paths. At aligned outputs, it keeps the more general comparable type. Later code may use only a type guaranteed by both paths. This opposite treatment of inputs and outputs is called *variance*. An output index survives only if both paths preserve the same input-to-output relationship. A branch-dependent permutation is therefore forgotten.

Displayed effects may have unequal depths because both omit an unchanged frame. Suppose p shows k more input cells than q . A merge is considered only if p also shows exactly k more output cells. The missing k -cell prefix of q is treated as unchanged, after which active suffixes are aligned. Formally, a necessary shape condition is

$$|L_p| - |L_q| = |R_p| - |R_q|.$$

A different net stack change or an incomparable aligned type rejects the join.

Write $\llbracket P \rrbracket$ for the inferred effect of a source segment P . For an **IF**, the declared selector effect is composed separately:

$$\llbracket \text{IF } P \text{ ELSE } Q \text{ FI} \rrbracket = (\text{flag} - -) \cdot (\llbracket P \rrbracket \sqcap \llbracket Q \rrbracket).$$

When **ELSE** is absent, Q is the identity effect.

3.3 Finite repetition

For a repeated phrase with one-pass effect p , the prototype computes

$$\pi_2(p) = p \sqcap (p \cdot p),$$

provided both the composition and common-contract operations are defined. This one-versus-two comparison is finite and terminating. It detects many bodies that grow, shrink, or change the stack incompatibly on a second pass.

The notation has a narrow meaning. It does not summarize every possible iteration count. One and two passes can agree after information has been lost. A later pass may then remove another correlation. A full loop analysis would propagate the loop-header state until that state stops changing. π_2 does not perform that process. It also proves neither termination nor properties of concrete values. It is only a local stack-stability test.

4 Evaluator Inputs

The run uses three files: a type profile, a word profile, and the example source. Appendices A–C reproduce every line from those files that affects the nine definitions. Table 1 summarizes the primitive contracts. The integer-literal entry classifies tokens such as 0, 1, and 5. It does not evaluate their values.

Class	Word	Declared effect
Literals	integer, FALSE	(- n), (- flag)
Stack	DUP	(x[1] - x[1] x[1])
	SWAP	(x[2] x[1] - x[1] x[2])
	OVER	(x[2] x[1] - x[2] x[1] x[2])
	ROT	(x[3] x[2] x[1] - x[2] x[1] x[3])
Numeric	+, -	(n n - n)
Logic	OR	(flag flag - flag)
	INVERT	(flag - flag)
Memory	@	(a-addr - x)
	C@	(c-addr - char)
Input	KEY	(- char)
Control	IF, WHILE, UNTIL	(flag -)

Table 1: Primitive effects used by the examples.

The presentation shows $0=$ as (n - flag). The word profile in Appendix C uses the broader effect (x - flag). This difference does not change a reported result. In `test8`, `KEY` produces `char`. The relation `char` $\preceq$ `n` $\preceq$ `x` makes both declarations applicable.

The word profile also describes control syntax. Angle brackets name captured source segments. Square brackets mark an optional segment. Effect-recipe lines are composed from top to bottom. A bare control word contributes its declared effect. `either` applies the common-contract operation $\sqcap$. `repeat` applies the finite operation π_2 .

The conditional declaration is:

```
syntax: IF <then branch> [ELSE <else branch>] FI
effect:
  IF
  either <then branch> <else branch>
```

For loops, the relevant recipes are:

```
syntax: BEGIN <loop prefix> WHILE <loop body> REPEAT
effect:
  repeat <loop prefix> WHILE
  repeat <loop body>
```

```
syntax: BEGIN <loop body> UNTIL
effect:
  repeat <loop body> UNTIL
```

The syntax declarations split source into segments. The effect recipes then combine those segments with the three operations already defined.

The exact definitions were checked by loading the three files. The resulting word effects were read from the definition log. One reproducible invocation is:

```
./run-evaluator-gforth.sh --prog examples.txt
```

Equivalently, after compiling the Java sources:

```
javac evaluator/*.java
java evaluator.Evaluator \
  --types forth2012types.txt \
  --specs forth2012specs.txt \
  --prog examples.txt
```

All nine definitions are accepted. Table 2 reproduces their inferred contracts. The next sections derive each result.

Word	Inferred effect	Principal mechanism
test1	(- n)	numeric sequence
test2	(flag flag - flag flag)	flag propagation
test3	(a-addr[2] x[1] - x[1] a-addr[2] x)	identity and backward refinement
test4	(n flag - n)	implicit identity branch
test5	(x[1] flag - x[1] x)	output widening
test6	(x x a-addr flag - x x x)	path-dependent permutation loss
test7	(flag flag - flag flag)	two-part repetition
test8	(-)	balanced test loop
test9	(a-addr flag - x)	input/output variance

Table 2: Exact inferred effects recorded in `examples.txt.log`.

5 Straight-Line Examples

5.1 Test1: stack shape without value evaluation

```
: test1 3 4 5 - + ;
```

Every literal has effect (- n). Naming fresh abstract results a , b , and c , the stack evolves as

$$\epsilon \longrightarrow n_a \longrightarrow n_a n_b \longrightarrow n_a n_b n_c \longrightarrow n_a n_d \longrightarrow n_e.$$

Subtraction consumes n_b, n_c and produces a fresh n_d . Addition consumes n_a, n_d and produces n_e . No input is required. One number is left:

`test1 ( - - n ).`

Concrete execution would calculate $4 - 5 = -1$ and then $3 + (-1) = 2$. The checker does not perform that calculation. Its domain contains the type `n`, but no integer constants or arithmetic expressions. It establishes only (- n).

5.2 Test2: a generic shuffle specialized to flags

```
: test2 OR FALSE SWAP ;
```

`OR` first requires two flags and leaves one fresh flag. `FALSE` adds another flag. The generic inputs of `SWAP` then match these two results. Both inputs are refined from `x` to `flag`:

$$\text{flag}_a \text{flag}_b \xrightarrow{\text{OR}} \text{flag}_c \xrightarrow{\text{FALSE}} \text{flag}_c \text{flag}_d \xrightarrow{\text{SWAP}} \text{flag}_d \text{flag}_c.$$

The inputs are consumed by `OR`; neither output is correlated with an original input. The reported interface therefore needs no visible indices:

`test2 (flag flag - flag flag).`

Although one output is produced by the word named `FALSE`, the checker has no constant-propagation domain and promises only its declared type.

5.3 Test3: backward refinement through SWAP and DUP

```
: test3 SWAP DUP @ ;
```

Let the initial suffix be $A B$, with B topmost. **SWAP** produces $B A$ and **DUP** produces $B A A$. The final **@** requires the top copy of A to be an **a-addr**. Because both copies carry the same symbolic index, the substitution also refines the untouched copy and the original input:

$$A B \xrightarrow{\text{SWAP}} B A \xrightarrow{\text{DUP}} B A A \xrightarrow{@} B A X_{\text{fresh}}.$$

After normalized numbering, A is **a-addr[2]** and B is **x[1]**. Hence

```
test3 (a-addr[2] x[1]
      - x[1] a-addr[2] x).
```

This is the key identity-flow example. Tracking only stack depth would miss the address requirement. Tracking types without indices would also lose information. Such an analysis could refine only the consumed copy. It could not identify the surviving copy as the same aligned address.

6 Branch Examples

6.1 Test4: an explicit effect versus an implicit frame

```
: test4 IF 1 + FI ;
```

The true branch $1 +$ pushes an **n**, then needs one additional **n** below it for $+$. Its inferred effect is $(\mathbf{n} - \mathbf{n})$. The absent **ELSE** branch has the identity effect $(-)$. These displayed depths differ, but the empty branch preserves an arbitrary deeper frame. Making one frame cell explicit gives the alignment

```
true  : (n - n),
false : (n - -n).
```

Both paths preserve a one-cell numeric stack shape. They do not preserve the same value. The false path returns the original number. The true path returns the result of addition. No output identity is claimed. After the leading **IF** consumes its selector, the complete definition is

```
test4 (n flag - n).
```

This example explains why comparing only the explicitly printed sides would be too strict. Stack effects denote transformations under an implicit frame.

6.2 Test5: a common prefix survives, a branch-dependent identity does not

```
: test5 IF DUP ELSE 0 FI ;
```

The true branch is $(\mathbf{x}[1] - \mathbf{x}[1] \mathbf{x}[1])$. The false branch explicitly has $(- \mathbf{n})$, but it leaves the deeper frame unchanged. Exposing one frame cell aligns the branches as follows:

```
true  : (x[1] - x[1] x[1]),
false : (x[1] - x[1] n).
```

The lower output is the original input on both paths, so its index survives. The top output is the same input on the true path but a fresh numeric literal on the false path. Its path-independent type is the more general type **x**. It cannot retain the input index. The branch contract is thus

```
(x[1] - x[1] x).
```

Composing the selector yields

```
test5 (x[1] flag - x[1] x).
```

The result loses two facts. First, `n` is widened to `x` at the differing output. Second, a correlation valid on only one path is discarded.

6.3 Test6: compatible shape with no common permutation

```
: test6 IF ROT ELSE @ FI ;
```

Write the pre-branch suffix as $A\ B\ C$, with C topmost. The true branch rotates it:

$$A\ B\ C \mapsto B\ C\ A.$$

The false branch `@` displays only one input and one output. Its implicit two-cell frame makes the comparable transformation

$$A\ B\ C \mapsto A\ B\ D,$$

where C must be `a-addr` and D is a fresh `x`. At the input, `ROT`'s generic requirement and `@`'s address requirement meet at the more specific `a-addr`. The input contract is consequently $(\ x\ x\ \text{a-addr} - \dots\)$.

The output positions have the path pairs (B, A) , (C, B) , and (A, D) . None of these establishes one common source identity. Their only common output type is `x`, so the branch contract is

$$(\ x\ x\ \text{a-addr} - x\ x\ x).$$

After selector consumption, the inferred definition is

```
test6 (x x a-addr flag - x x x).
```

The checker accepts the equal net stack change. It forgets which input reaches which output. An analysis that stores each path separately could retain both permutations. This checker stores one common contract.

6.4 Test9: input and output variance in isolation

```
: test9 IF @ ELSE C@ FI ;
```

The two branch effects are

$$(\text{a-addr} - x) \quad \text{and} \quad (\text{c-addr} - \text{char}).$$

An input to the whole conditional must satisfy both address requirements. Since $\text{a-addr} \preceq \text{c-addr}$, the common input is the more specific `a-addr`. An output client, however, needs a guarantee valid for either `x` or `char`. Since $\text{char} \preceq x$, that guarantee is the more general `x`. Thus

$$(\text{a-addr} - x) \sqcap (\text{c-addr} - \text{char}) = (\text{a-addr} - x),$$

and selector composition gives

```
test9 (a-addr flag - x).
```

This small example is the clearest demonstration of contract variance: branch inputs are narrowed, whereas outputs are widened.

7 Repetition Examples

7.1 Test7: one-versus-two exposes both flag requirements

```
: test7 BEGIN SWAP OVER WHILE INVERT REPEAT ;
```

The profile divides this structure into the repeated prefix `SWAP OVER WHILE` and the repeated body `INVERT`. Consider one prefix pass on $A\ B$:

$$A\ B \xrightarrow{\text{SWAP}} B\ A \xrightarrow{\text{OVER}} B\ A\ B \xrightarrow{\text{WHILE}} B\ A.$$

`WHILE` consumes the duplicate of the original top cell, so B must be a flag. A second prefix pass transforms $B\ A$ back to $A\ B$ and tests A . Consequently the two-pass effect requires both original cells to be flags. The common summary preserves two flag-typed positions. It cannot promise one permutation. One pass swaps the cells, while two passes restore them. The summary is therefore

$$(\text{flag flag} - \text{flag flag}).$$

The second repeated region, `INVERT`, has the stable type effect $(\text{flag} - \text{flag})$. Composing it with the prefix summary leaves the same two-cell interface, giving

$$\text{test7}(\text{flag flag} - \text{flag flag}).$$

This is a useful rejection test for changing stack shape and incompatible second-pass types. It is not a proof that the reported identity-free summary characterizes every number of loop iterations or that the loop terminates.

7.2 Test8: a data-stack-neutral loop pass

```
: test8 BEGIN KEY 0= UNTIL ;
```

A complete repeated pass has the data-stack trace

$$\epsilon \xrightarrow{\text{KEY}} \text{char} \xrightarrow{0=} \text{flag} \xrightarrow{\text{UNTIL}} \epsilon.$$

The displayed numeric contract for `0=` accepts `char` because $\text{char} \preceq \text{n}$. The broader `x` contract also accepts it. `UNTIL` consumes the resulting flag. One pass has the identity data-stack effect. Two passes have the same effect. Their common contract is

$$\text{test8}(-).$$

This conclusion concerns only the modeled data stack. `KEY` still has an I/O side effect. The result does not promise a key that meets the exit condition. It also does not prove termination.

8 Discussion

8.1 What the examples establish

The nine examples isolate complementary properties of the symbolic domain. `test1` shows that the checker infers shape and cell type without constant evaluation. `test2` shows subtype specialization of a generic stack word. `test3` demonstrates bidirectional information flow: execution order is left to right, but a late requirement propagates backward through shared symbolic identities.

The branch examples show why an effect is more than a pair of stack heights. `test4` depends on the implicit frame. `test5` retains only an identity common to both paths. `test6` loses a branch-dependent permutation. `test9` narrows its input and widens its output. The loop examples show

two uses of the finite rule. A second pass refines another cell in `test7`. A balanced pass becomes identity in `test8`.

All reported outputs are inferred from primitive contracts. The illustrative comments in the presentation are not declarations. This rule prevents a stale comment from changing inference. Semantic labels such as `value` or “address contents” are therefore outside the domain. A richer profile would be needed to represent them.

8.2 Configuration as an analysis parameter

The same source may receive a different verdict under another valid profile. For example, one profile may classify `char` as numeric. It accepts the displayed `KEY 0=`. Another profile may keep `char` and `n` unrelated. A numeric-only `0=` then rejects the phrase. The target Forth system determines which policy is appropriate. The profile makes that policy reviewable.

External control declarations provide similar flexibility. They also enlarge the trusted base. A recipe might omit a selector effect. It might also put `UNTIL` outside its repeated segment. Either error can produce a wrong summary even when the three operations are correct. The configuration should therefore be reviewed with the checked code.

9 Validity and Limitations

Acceptance has a precise but bounded interpretation:

- every checked linear boundary has enough modeled cells of comparable types;
- every checked branch admits one common contract under the implemented alignment and variance rules;
- every checked repetition admits the finite $p \sqcap (p \cdot p)$ summary;
- parser roles, delimiters, and compilation state fit the active profile.

It does not imply full Forth program correctness. The principal threats to a stronger interpretation are as follows.

Trusted declarations. Primitive and parser specifications are assumptions. If `@`, `KEY`, or a control word is declared incorrectly, analysis of its clients cannot make the result reliable.

Single effects and comparable types. The dictionary stores one effect per primitive. Branch merging also returns one common contract. It cannot retain a separate permutation for each path. Aligned types must be directly comparable. The checker does not search for a third common type.

Finite loop approximation. The one-versus-two comparison does not prove a result for every iteration count. A stronger analysis would maintain an abstract stack at the loop header. It would propagate the body until that stack stopped changing. Another design would require a declared loop interface and check every back edge against it [1].

Forth language coverage. Return-stack manipulation, unrestricted `EXECUTE`, recursion, non-local `THROW`, native defining words, self-modifying dictionaries, and unrestricted metaprogramming require additional abstractions. The checker should complement runtime tests, code review, and target-system knowledge.

The examples are explanatory cases. They are not a performance benchmark. They also do not prove that every accepted program is safe. Each case makes one operation visible. The exact nine contracts can be compared across the Java and Gforth implementations.

10 Related Work

Forth has long used stack-effect notation as an interface convention. The Forth standard defines data types and control semantics [2]. It does not make each source comment a static judgment. Earlier work developed algebraic composition and multiple effects for control flow [4, 5]. Later work added typed symbolic variables and conservative control-flow operations [6, 7]. A Java framework made these ideas executable [8]. The checker described here also loads parser and control behavior from a profile.

Stoddart and Knaggs also studied type inference for stack languages [10]. Saabas and Uustalu gave compositional type systems for stack-based low-level code [9]. JVM bytecode verification also propagates stack types through control-flow joins [3]. Those settings have more fixed instruction and control languages. Forth source can contain parser words and dictionary extensions. The profile handles that variability, but it becomes part of the trusted input.

Abstract interpretation offers a stronger treatment of loops [1]. It propagates abstract states until they stop changing. The one-versus-two rule performs only the first bounded comparison. An inferred loop effect must be read with that distinction in mind. Every bibliography entry below includes a public full-text link. The links were checked on 30 August 2026.

11 Conclusion

Typed symbolic stack effects turn a familiar Forth notation into executable, configurable documentation. Sequential composition checks touching stack boundaries and propagates refined types through shared identities. Branch merging computes one conservative contract, narrowing inputs, widening outputs, and preserving only path-independent correlations. Repetition compares one pass with two to obtain a useful but explicitly finite stability summary.

The nine examples make these mechanisms concrete. Three linear definitions show type and identity propagation. Four conditionals show implicit frames, input narrowing, output widening, and information loss. Two loops show what the bounded repetition rule accepts. The profile reproduces every inferred effect. The result is a practical checker for the configured subset of Forth. It is not a proof of functional behavior or termination. Stronger assurance requires validated primitives, separate path effects, preserved identity classes, and full loop analysis.

A Example Source and Inferred Log

The source file used for Table 2 is:

```
: test1 3 4 5 - + ;
: test2 OR FALSE SWAP ;
: test3 SWAP DUP @ ;
: test4 IF 1 + FI ;
: test5 IF DUP ELSE 0 FI ;
: test6 IF ROT ELSE @ FI ;
: test7 BEGIN SWAP OVER WHILE INVERT REPEAT ;
: test8 BEGIN KEY 0= UNTIL ;
: test9 IF @ ELSE C@ FI ;
```

The generated definition log is:

```
test1 ( -- n )
test2 ( flag flag -- flag flag )
test3 ( a-addr[2] x[1] -- x[1] a-addr[2] x )
test4 ( n flag -- n )
test5 ( x[1] flag -- x[1] x )
test6 ( x x a-addr flag -- x x x )
test7 ( flag flag -- flag flag )
test8 ( -- )
test9 ( a-addr flag -- x )
```

Whitespace in the machine log is presentation-only; the symbolic contracts are as shown.

B Type-Profile Declarations

The excerpt below contains the types, aliases, and subtype edges needed by the nine examples. It is copied from `forth2012types.txt`. Unused profile types, such as execution tokens, file identifiers, and double-cell numbers, are omitted.

```
type x X cell
type n number signed
type +n nonnegative
type u unsigned
type char c
type flag boolean
type addr
type c-addr ca
type a-addr aa
```

```
rel +n < n
rel +n < u
rel u < n
rel n < x
rel u < x
rel char < +n
rel flag < x
rel addr < u
```

```
rel c-addr < addr
rel a-addr < c-addr
```

The loader adds every relation implied by a chain. For example, it derives `char` $\preceq$ `n` $\preceq$ `x`. It also derives `a-addr` $\preceq$ `c-addr` $\preceq$ `x`. The type `flag` remains outside the numeric chain.

C Word and Control Specifications

This appendix selects the entries from `forth2012specs.txt` used to parse and infer `examples.txt`. The text of each declaration is unchanged; unrelated vocabulary entries are omitted.

C.1 Stack words, definition syntax, and literals

```
OVER ( x[2] x[1] -- x[2] x[1] x[2] )
SWAP ( x[2] x[1] -- x[1] x[2] )
DUP ( x[1] -- x[1] x[1] )
ROT ( x[3] x[2] x[1] -- x[2] x[1] x[3] )

: parse word define colon ( -- )
; control end ( -- )
literal integer ( -- n )
FALSE ( -- flag )
```

The first four declarations expose symbolic identity. The source-reading markers on `:` and `;` start and close a colon definition. The literal class supplies the effect of tokens such as `0`, `1`, and `5`.

C.2 Conditional control structure

```
IF control if ( flag -- )
ELSE control else ( -- )
FI control fi ( -- )

syntax: IF <then branch> [ELSE <else branch>] FI
effect:
  IF
  either <then branch> <else branch>
```

The control roles delimit the captured source regions. The effect recipe first accounts for selector consumption and then applies the common-contract operation to the two branches.

C.3 Loop control structures

```
BEGIN control begin ( -- )
WHILE control while ( flag -- )
REPEAT control repeat ( -- )

syntax: BEGIN <loop prefix> WHILE <loop body> REPEAT
effect:
  repeat <loop prefix> WHILE
  repeat <loop body>
```

```
UNTIL control until ( flag -- )
```

```
syntax: BEGIN <loop body> UNTIL
effect:
  repeat <loop body> UNTIL
```

These recipes identify exactly which control-word effects belong to the one-versus-two calculation. In particular, `WHILE` is included with the loop prefix and `UNTIL` with the loop body.

C.4 Numeric, logical, memory, and input words

```
+ ( n n -- n )
- ( n n -- n )
0= ( x -- flag )
OR ( flag flag -- flag )
INVERT ( flag -- flag )

@ ( a-addr -- x )
C@ ( c-addr -- char )
KEY ( -- char )
```

This profile lets `0=` accept `x`. The presentation uses the narrower input type `n`. Section 4 explains why both choices give the same result here.

References

- [1] P. Cousot and R. Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of POPL*, pp. 238–252, 1977. doi:10.1145/512950.512973. Public full text: <https://www.di.ens.fr/~cousot/publications.www/CousotCousot-POPL-77-ACM-p238--252-1977.pdf>.
- [2] Forth 200x Standardisation Committee. *Forth 2012 Standard*. Draft proposed standard, 10 November 2014. Public full text: <https://forth-standard.org/standard/intro>.
- [3] X. Leroy. Java bytecode verification: Algorithms and formalizations. *Journal of Automated Reasoning*, 30:235–269, 2003. doi:10.1023/A:1025055424017. Public full text: <https://xavierleroy.org/publi/bytecode-verification-JAR.pdf>.
- [4] J. Pöial. Algebraic specification of stack effects. *Forth Dimensions*, 16(4):18–20, 1994. Public full text: <https://www.forth.org/fd/FD-V16N4.pdf>.
- [5] J. Pöial. Multiple stack-effects of Forth programs. In *1991 FORML Conference Proceedings, euroFORML'91*, pp. 400–406, 1992. Public full text: <https://www.complang.tuwien.ac.at/anton/euroforth/ef91/poial.pdf>.
- [6] J. Pöial. Stack effect calculus with typed wildcards, polymorphism and inheritance. Presentation and proceedings abstract, 18th EuroForth Conference, 2002. Public full text: <https://www.complang.tuwien.ac.at/anton/euroforth/ef02/poial02.pdf>.
- [7] J. Pöial. Typing tools for typeless stack languages. In *Proceedings of the 22nd EuroForth Conference*, pp. 40–46, 2006. Public full text: <https://www.complang.tuwien.ac.at/anton/euroforth/ef06/poial06.pdf>.
- [8] J. Pöial. Java framework for static analysis of Forth programs. In *Proceedings of the 24th EuroForth Conference*, pp. 20–23, 2008. Public full text: <https://www.complang.tuwien.ac.at/anton/euroforth/ef08/papers/poial.pdf>.
- [9] A. Saabas and T. Uustalu. Compositional type systems for stack-based low-level languages. In *Proceedings of CATS 2006*, CRPIT 51, pp. 27–39, 2006. Public full text: https://web.archive.org/web/20120325100058id_/http://crpit.com/confpapers/CRPITV51Saabas.pdf.
- [10] B. Stoddart and P. J. Knaggs. Type inference in stack based languages. In *1991 FORML Conference Proceedings, euroFORML'91*, pp. 407–421, 1991. Public full text: <https://www.complang.tuwien.ac.at/anton/euroforth/ef91/stoddart.pdf>.